

10 Users

GIZMo-Kria is configured to automatically startup. If the monitor is being attached to a detector for the first time, a user will need to conduct an auto-calibration for the system to interpolate data during normal operation.

10.1 Connections

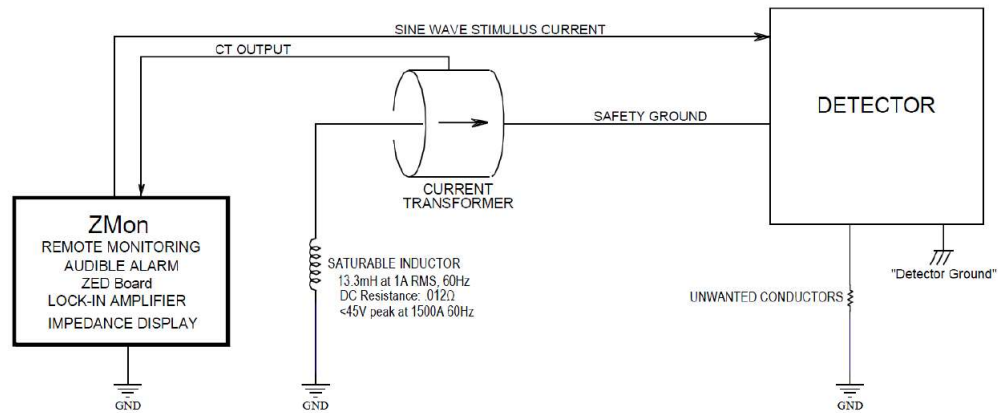


Figure 154: Impedance Monitor Connections

1. Connect the impedance monitor's Stimulus Out BNC signal-positive to the detectors grounding point.
2. Connect the shield of the Stimulus Out BNC to building ground
3. Connect a BNC to the CT IN port on the back of the impedance monitor, connect the opposite end to the current transformer inside of the Saturable Inductor (SI) enclosure.
4. Connect a 12V power supply to the back of the impedance monitor
5. Connect a USB cable to the back of the impedance monitor
6. Connect a CAT5e or CAT6 ethernet cable to ETH3 or ETH4 on the back of the impedance monitor
7. Connect the amber alert beacon to the back panel
8. Connect the audio amplifier speaker output to the back panel

10.2 Log In

There are multiple ways to interact with the impedance monitor. The following section describes how to use USB, SSH, and ZMQ or OPC-UA with GIZMo-Kria. The ZMQ and OPC-UA clients are the preferred method for users to interact with the system. There is limited functionality for users running the client scripts on the impedance monitor instead of from a connected device.

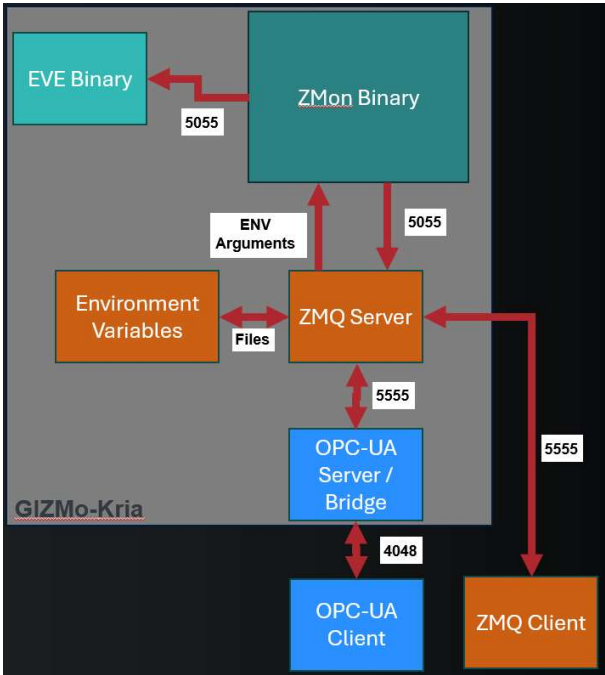


Figure 155: Basic Software Block Diagram

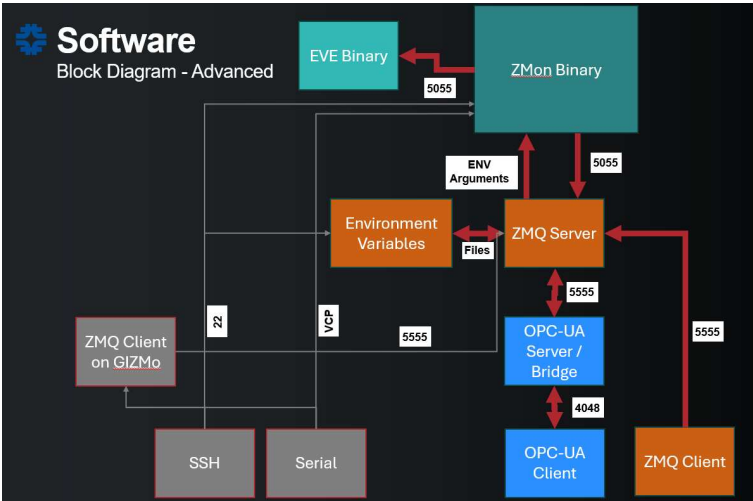


Figure 156: Advanced Software Block Diagram

10.2.1 USB

MS Windows Instructions:

Download and install a serial terminal monitor such as PuTTY.

www.putty.org

Open Putty and device manager. Plug in the USB cable to your machine and determine which COM port GIZMo-Kria's USB connection has enumerated as (example: COM101).

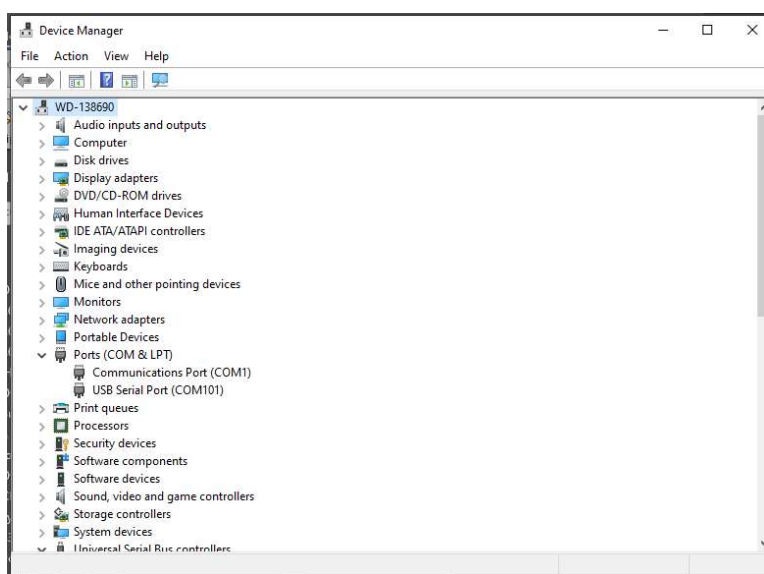


Figure 157: Device Manager

In PuTTY, select the Serial radial button, enter the COM Port number into the Serial Line Box, then enter the BAUD rate (115200) into the Speed Box.

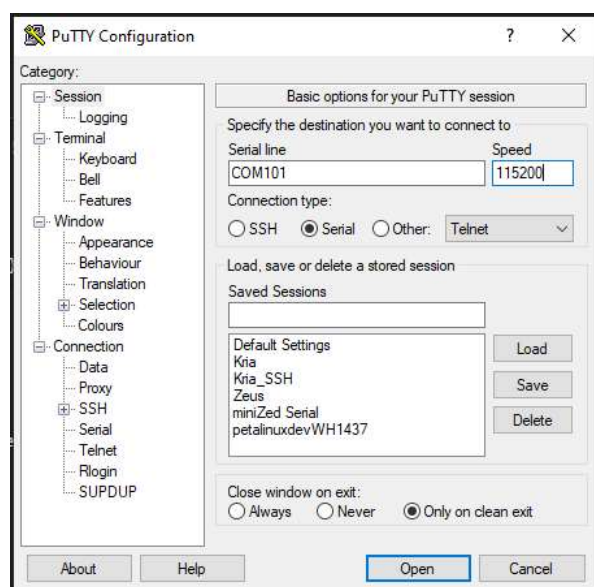


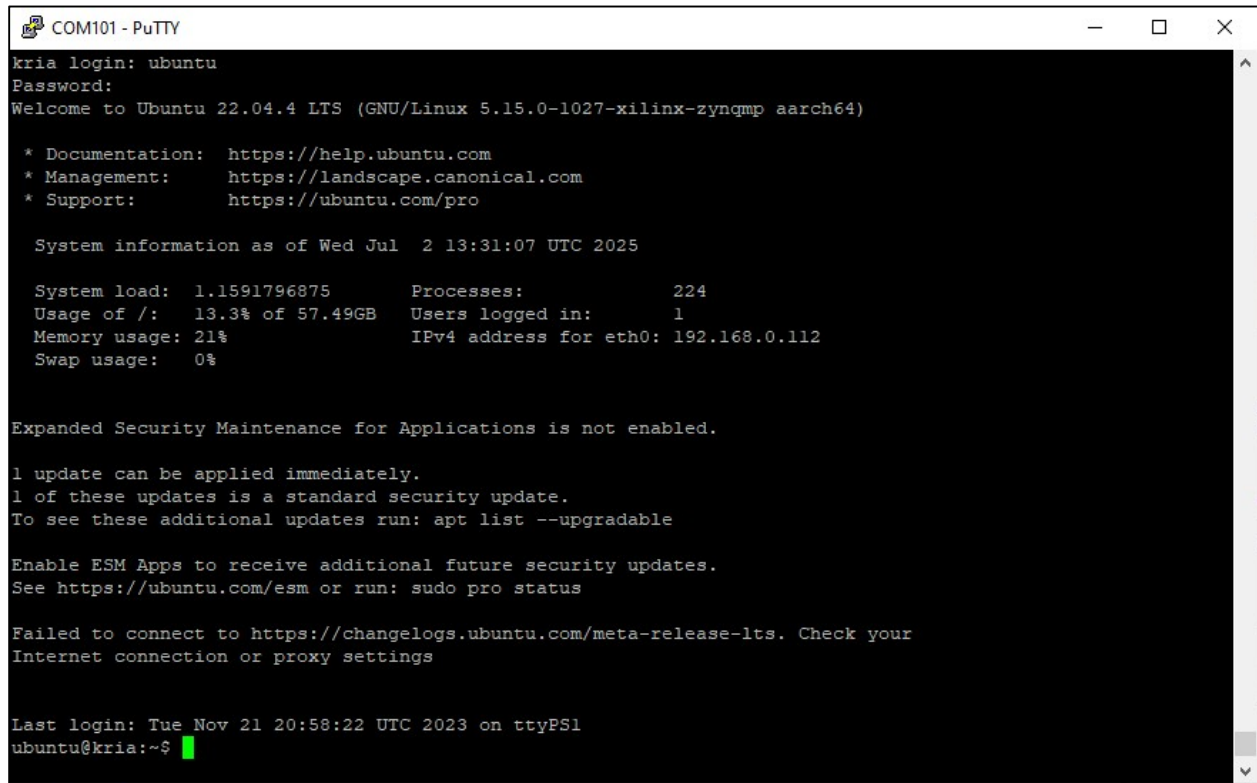
Figure 158: PuTTY configuration

Select the Open button in PuTTY, this should open a new serial session with the Impedance Monitor. If the screen is black, press enter to be prompted with the login.

Enter the username and password to login.

Username: ubuntu

Password: fermilabGIZMO



```
COM101 - PuTTY
kria login: ubuntu
Password:
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-1027-xilinx-zynqmp aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jul  2 13:31:07 UTC 2025

System load:  1.1591796875      Processes:           224
Usage of /:   13.3% of 57.49GB   Users logged in:    1
Memory usage: 21%              IPv4 address for eth0: 192.168.0.112
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your
Internet connection or proxy settings

Last login: Tue Nov 21 20:58:22 UTC 2023 on ttyPS1
ubuntu@kria:~$
```

Figure 159: PuTTY Terminal Window

Run the ZMQ client

```
ubuntu@kria:~$ python3 zmqClient.py
```

Running the ZMQ client on the impedance monitor has less functionality than running from a connected device. With the limited version, users can set_time, clear_latch, get_data, set run and calibration measurement amounts, set_th, read_adc, and run testHello.py.

```

COM101 - PuTTY
ubuntu@kria:~$ python3 zmqClient.py
Enter a command:

set_time          --> Set the time manually or from the local clock
clear_latch       --> Clear the latched state of the Impedance Monitor
get_data          --> Read Impedance Monitor Socket Data
get_data_continuous --> Continuously Read Impedance Monitor Socket Data
run N             --> Run with N seconds between each measurement
CAL N            --> Calibrate the system with N/10 seconds between each measurement
get_Rcal          --> Get the Rcalibration.csv file from the Impedance Monitor
get_Ccal          --> Get the Ccalibration.csv file from the Impedance Monitor
plot_Rcal         --> Plot the Rcalibration.csv file after retrieving from the the Impedance Monitor
fit_curve         --> Fit an exponential curve to the calibration data
interpolate       --> Conduct a reverse interpolation using the measured magnitude
set_th N          --> Set the impedance (N) threshold for the system alarm
read_adc          --> Read the ADC in the Impedance Monitor
get_adc           --> Get the adc.csv file after reading the ADC
plot_adc          --> Plot the adc.csv file after getting it from the Impedance Monitor
testHello.py      --> Run the testHello.py script on the Impedance Monitor
exit

> set_time
set time Manually or from Local clock?
M for Manually setting the time
L for use Local Clock
> m
Enter a Date - Time - String in the following format
YYYY-MM-DD HH:MM:SS
2025-07-07 12:53:00
Sending set_time 2025-07-07 12:53:00
Received:
System Time has been updated

```

Figure 160: zmqClient.py running on the Impedance Monitor

Linux Instructions:

Download and install 'screen' with dnf:

```
tom@linux:~$ sudo dnf install screen
```

Add your user to the dialout group:

```
tom@linux:~$ sudo usermod -aG dialout <username>
```

Replacing <username> with your username.

Determine which device number the impedance monitor is enumerated as:

```
tom@linux:~$ ls /dev | grep USB
```

```
[tdeline@woody ~]$ ls /dev | grep USB
ttyUSB0
ttyUSB1
ttyUSB2
ttyUSB3
```

The FTDI chip inside of the impedance monitor uses USB1 for UART.

Run the screen command, give the device location, and enter the BAUD rate:

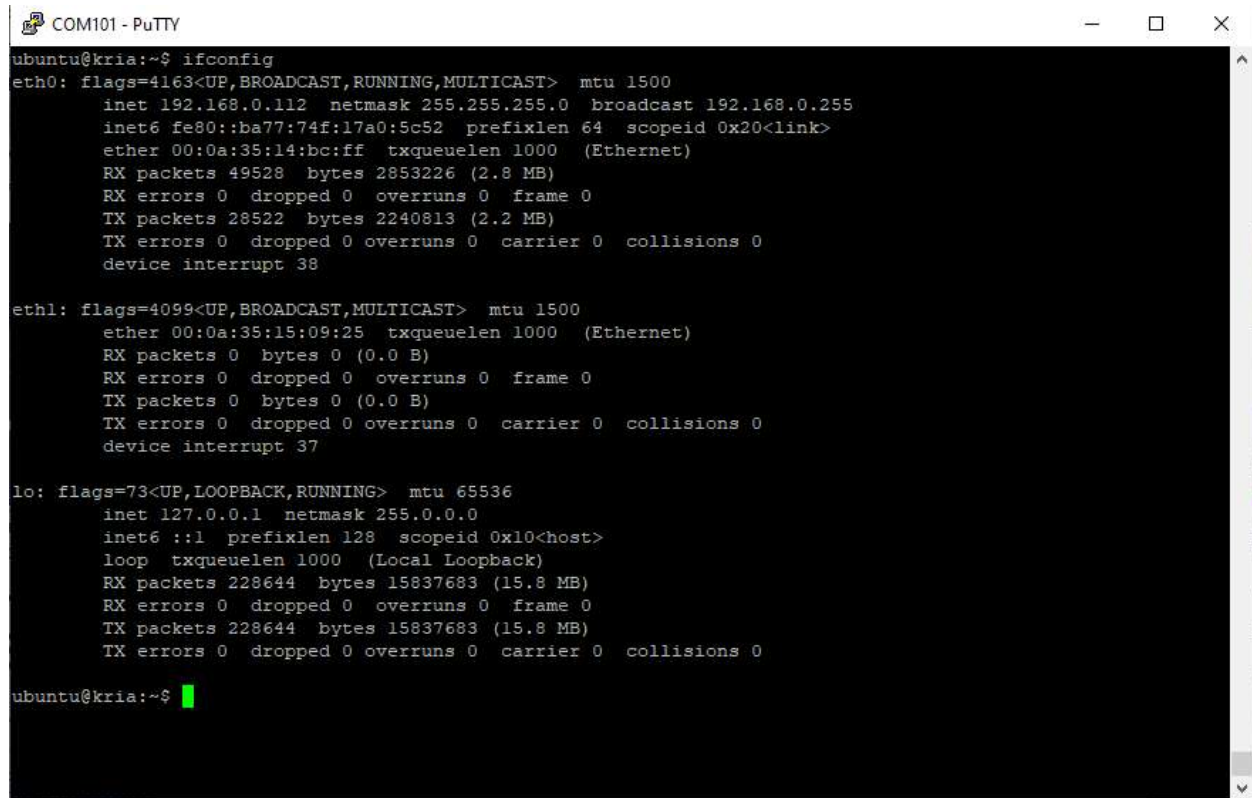
```
tom@linux:~$ screen /dev/ttyUSB1 115200
```

Run the zmqClient.py script as per the Windows section.

10.2.2 SSH

To connect to the system using SSH, first determine the impedance monitor's IP address. The device is configured to automatically display the IP address on the display when ethernet is connected with a DHCP connection. If no IP address is visible, connect a USB cable and log into the system as per the USB section of the Users document. Once logged in, run:

```
ubuntu@kria:~$ ifconfig
```



```

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.0.112  netmask 255.255.255.0  broadcast 192.168.0.255
    inet6 fe80::ba77:74f:17a0:5c52  prefixlen 64  scopeid 0x20<link>
    ether 00:0a:35:14:bc:ff  txqueuelen 1000  (Ethernet)
    RX packets 49528  bytes 2853226 (2.8 MB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 28522  bytes 2240813 (2.2 MB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
    device interrupt 38

eth1: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
    ether 00:0a:35:15:09:25  txqueuelen 1000  (Ethernet)
    RX packets 0  bytes 0 (0.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 0  bytes 0 (0.0 B)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
    device interrupt 37

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 228644  bytes 15837683 (15.8 MB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 228644  bytes 15837683 (15.8 MB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

ubuntu@kria:~$
```

Figure 161: Impedance Monitor Ethernet Ports

To change the IP address of the impedance monitor:

```
ubuntu@kria:~$ sudo ifconfig eth0 <desired.i.p.address>
```

10.2.3 ZMQ or OPC-UA Connected Clients

Connecting to the Impedance Monitor using ZMQ or OPC-UA client from a remote device is the desired method for connecting to, configuring, and reading the impedance monitor. Using a client device, first install python from python.org

<https://www.python.org/downloads/>

From the installer, be sure to check the box for “Add Python to PATH” at the bottom of the window and then select install.

With python installed, download the libraries for the ZMQ and OPC-UA clients. Open a powershell in windows and run:

```
PS U:\> pip install pyzmq

PS U:\> pip install opcua

PS U:\> pip install cryptography

PS U:\> pip install pandas

PS U:\> pip install numpy

PS U:\> pip install scipy

PS U:\> pip install matplotlib
```

With all library and dependencies installed, run the provided client script to connect with zmq or OPC-UA.

See ZMQ Client section for details on each command.

10.3 Setup

During initial system setup with GIZMo-Kria, the user should set the time on the machine. This can be done using the `zmqClient.py` script. Time is not persistent through re-boots and needs to be set each time the system loses power.

10.3.1 Setting the time

Enter the command `'set_time'` and enter `'L'` (not case-sensitive) to grab the time from your host machine and send it to the impedance monitor.

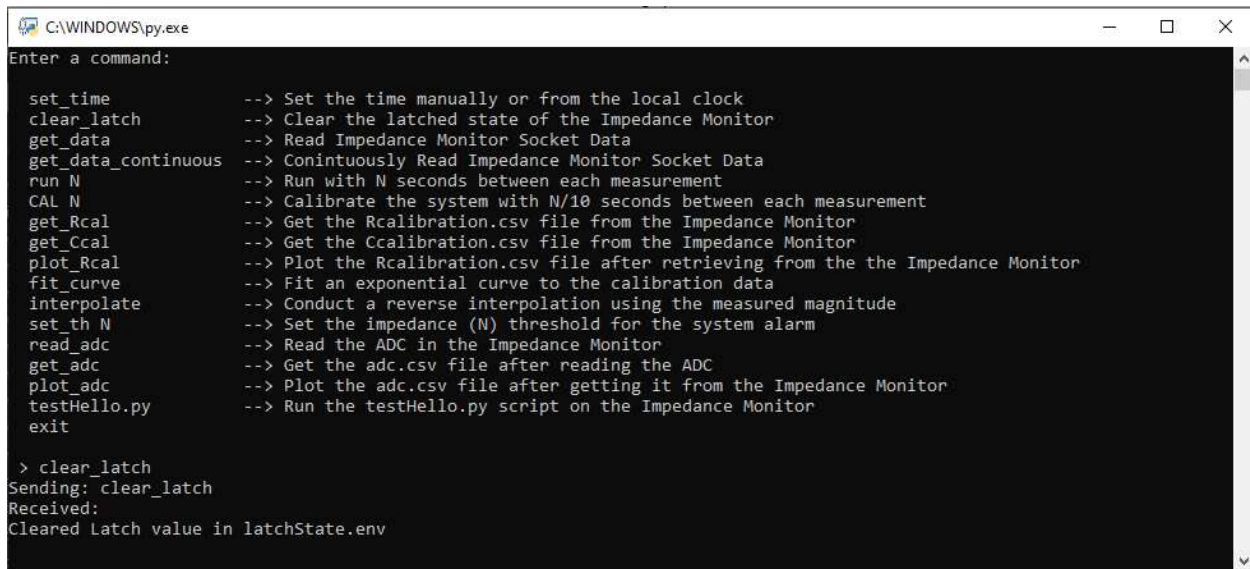
```
> set_time
set time Manually or from Local clock?
M for Manually setting the time
L for use Local Clock
> l
Sending set_time 2025-05-20 12:15:47.664134
Received:
System Time has been updated
```

Figure 162: set_time example

The display will go black and then return to normal operation. The `'System Time'` field on the display should reflect the current time. If the time is not correct, the time can manually be set using the `'M'` option instead of `'L'`.

10.3.2 Clearing the latch

After the time is set, clear any pre-existing latches from the system with the command `'clear_latch'`. This will wipe any timestamps from memory about when the system was triggered. If there is no timestamp on the `'System Latched at:'` field of the display, this step can be skipped.



```

C:\WINDOWS\py.exe
Enter a command:

set_time          --> Set the time manually or from the local clock
clear_latch       --> Clear the latched state of the Impedance Monitor
get_data          --> Read Impedance Monitor Socket Data
get_data_continuous --> Conintuously Read Impedance Monitor Socket Data
run N             --> Run with N seconds between each measurement
CAL N            --> Calibrate the system with N/10 seconds between each measurement
get_Rcal          --> Get the Rcalibration.csv file from the Impedance Monitor
get_Ccal          --> Get the Ccalibration.csv file from the Impedance Monitor
plot_Rcal         --> Plot the Rcalibration.csv file after retrieving from the the Impedance Monitor
fit_curve         --> Fit an exponential curve to the calibration data
interpolate       --> Conduct a reverse interpolation using the measured magnitude
set_th N          --> Set the impedance (N) threshold for the system alarm
read_adc          --> Read the ADC in the Impedance Monitor
get_adc           --> Get the adc.csv file after reading the ADC
plot_adc          --> Plot the adc.csv file after getting it from the Impedance Monitor
testHello.py      --> Run the testHello.py script on the Impedance Monitor
exit

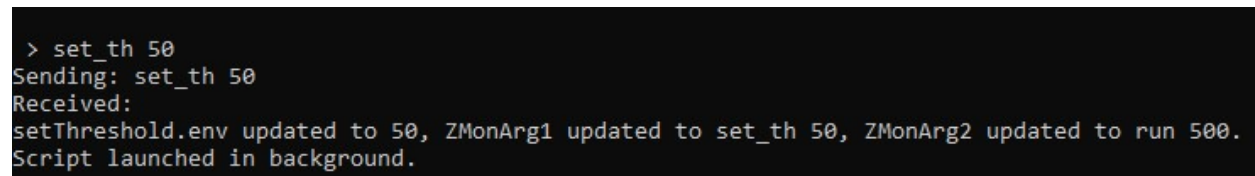
> clear_latch
Sending: clear_latch
Received:
Cleared Latch value in latchState.env

```

Figure 163: *clear_latch* example

10.3.3 Setting the alarm threshold

Then, set the threshold that the system should alarm at with the 'set_th' command



```

> set_th 50
Sending: set_th 50
Received:
setThreshold.env updated to 50, ZMonArg1 updated to set_th 50, ZMonArg2 updated to run 500.
Script launched in background.

```

Figure 164: *set_th to 50 ohms* example

10.3.4 Verifying connections

Next, check that the system is able to read the returned signal through the CT-In ADC with:

```
> read_adc
```

Download the results with:

```
> get_adc
```

And plot the results with:

```
> plot_adc
```

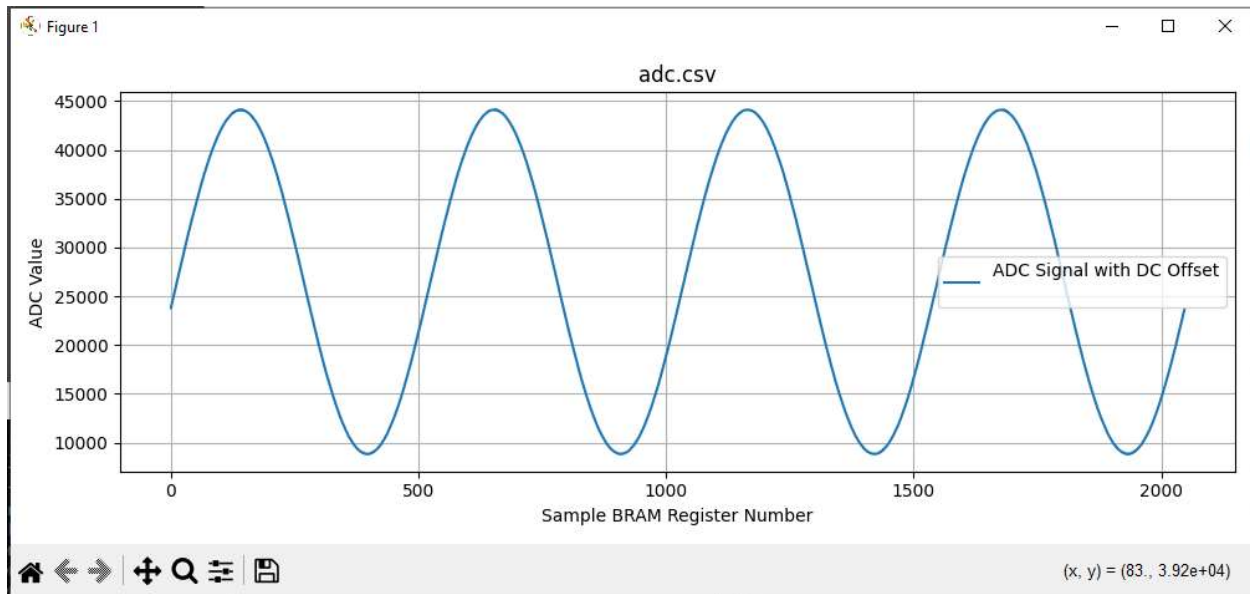


Figure 165: Plotting the ADC

10.3.5 Calibrate the system

The next step in configuring the Impedance Monitor is to calibrate the system. Using the ZMQ or OPC-UA client, enter the command 'CAL' and specify the number of measurements that the ZMon software should execute for each measurement. The system takes approximately one second per thousand measurements.

```
> CAL 1000
```

The display will go black during calibration and stay dark until calibration is completed. There are a total of 192 datapoints for the system to measure. Using 1000 measurements per data point will take around 3.5 minutes to execute. After the system is done calibrating, the display will become active again.

After calibrating, download the calibration data using the ZMQ client:

```
> get_Rcal
```

```
> get_Ccal
```

Use the built-in data plotting to view the results.

```
> plot_Rcal
```

```
> plot_Ccal
```

Results can also be viewed as a CSV file with Excel or similar. The file is located in the same directory as the client script.

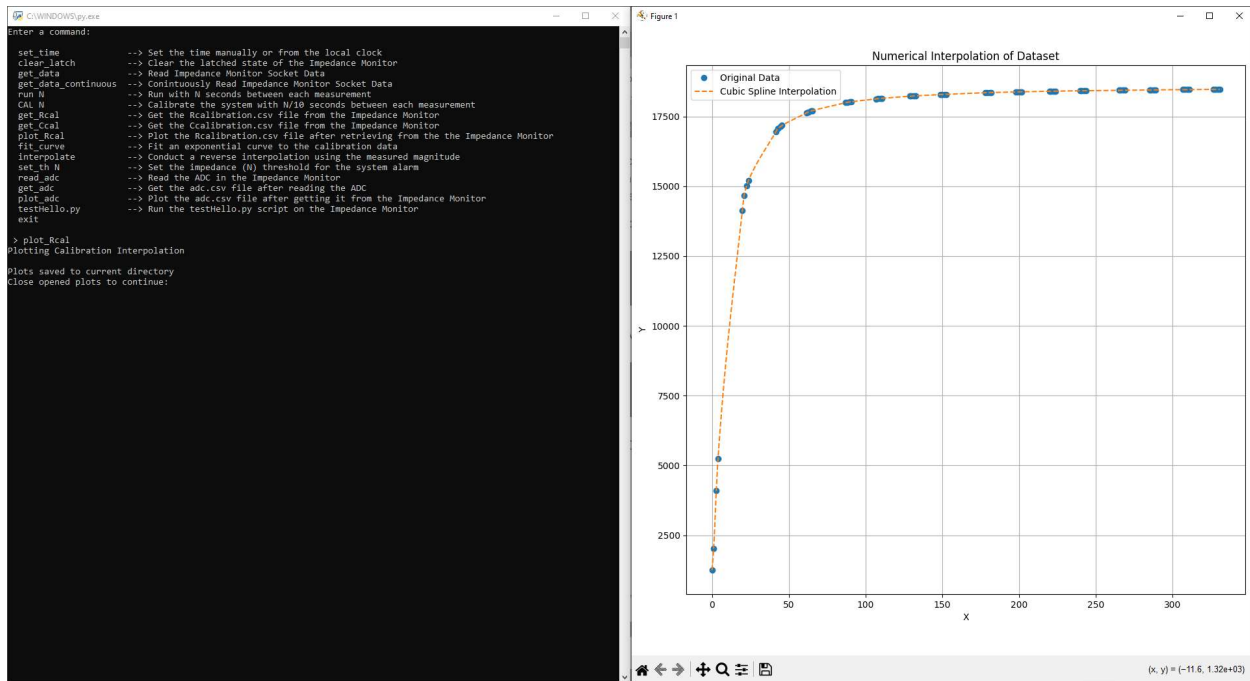


Figure 166: Resistor Calibration Data

10.3.6 Validate the results

Validate the results of the calibration run by shorting a resistor between Stimulus Out (+) and Stimulus Out (-). The first value should be around 100 ohms, and the display should read 100.0 (+/- 5ohms) in the display's Res field. Next test 55 ohms, then 0 ohms. When 0 ohms is applied, the system will latch and apply a timestamp to the latch field on the display. The latch time is also available in the data stream when using the 'get_data' command. The display should turn from green to red, the ground fault indicator LED on the front panel will illuminate, and the amber-flashing beacon along with the audio amplifier will activate.

Remove the shorted resistor connection, the system will keep track of the time that the short was first detected and the display will change from red to green. The fault LED, beacon and audio amplifier should also deactivate.

Validate the capacitance alarm by connecting a capacitor decade box in parallel with the resistor decade box. Add 40nF of capacitance and the system should display a capacitance alarm. Add 20uF of capacitance and the system should alarm the same as a resistive short would.

After validating the results, clear the latch on the system again with

```
> clear_latch
```

The system is now ready for continued use. Data should be continuously logged from a slow-controls system. If necessary, a program can be provided to continuously log data output from the system.